CS408 – Progress Report

Digital Logic Simulator

Aims and Objectives

The primary goal of this project is to produce a piece of software that acts as a simulator for digital logic circuits. The project's secondary goal is to provide a set of lessons that will teach the user digital logic in a practical and interactive way. The final result of this project will be a tool that acts as an all-in-one practical guide on how to produce a very simple, Turing-complete computer, based on the von Neumann architecture. This tool will be targeted at both students and casual learners looking for an interactive introduction to computational logic. The program should be able to teach someone, who knows nothing or little about digital logic, step-by-step how to build increasingly complex digital devices that will be combined to produce a functioning computer. This will be purely in the realm of logic, without requiring the user to learn about electronic engineering in the process.

The logic simulator will provide a simple, grid-based environment in which users can place wires, logic gates, and circuit packages. The circuit packages and logic gates will be similar to semiconductor packages, using pins to represent the inputs and outputs of the packages. Wires will be used to connect these pins to carry signals between them. Wires will be organised by coloured layers, meaning that wires of differing color cannot directly pass signals between each other. This allows for the creation of complex, layered wiring, but with clear visual distinction between different layers.

Each lesson will provide the user with two circuit packages, one will provide a set of pre-defined inputs, the other will take a set of outputs. The purpose of these packages will be to test that the user's circuit works as expected. If the received outputs are correct, the user will be able to progress to the next lesson. Each lesson will also provide a briefing note, outlining the task and provide the user with the information required to complete the task.

The program will also provide a "sandbox" mode in which there is no set objective, and the user is free to experiment with the simulator.

Related Work

There are many other pieces of software that tackle this problem domain, at least in terms of the simulator. Some do this better than others, but the design space appears to be fairly well explored.

The first is Boolr, a Dutch high school project implemented in Javascript. The notable attraction of this program is its clean interface and visual style. Despite its sharp, modern look it had several problems. The main issue is with small cyclic circuits, such as flip-flops, which could oscillate under certain circumstances, toggling the flip-flop very rapidly and not being able to settle its state. Another issue with Boolr is the storage of circuits. The circuits are stored in an XML format. In a test build of a small 8-bit machine, the project file quickly bloated to several gigabytes in size. It is not known if this is a fault with the format itself, or simply poor use in the program's implementation. After a point, the file became so large that the program could no longer load it, and doing so would cause the program to lock.

The second piece of software we will look at is Digital, a digital logic simulator with a massive library of features, including a set of analysis tools, such as representing the circuit as a truth table or a Karnaugh map. It is a spiritual successor to Logisim and solved many of the issues associated with it, such as circuit settling to put the circuit in a stable state before simulating it. Digital touts impressive performance capabilities, estimating that a simple processor can be simulated with a 120kHz clock. The developer's description of some core software features will inform some of the design decisions made in this project, specifically the idea of an event based simulator. The developer does not go into detail about what is meant by "event based", but it inspires some attractive ideas about how to approach the problem of ensuring that the simulator's logic is consistent and correct.

Digital and Boolr share some common useful features, the main one being the ability for the user to abstract circuits into black-box circuit packages. This is an invaluable tool in building digital circuits, especially when building something with as many interdependent components as a computer.

Requirements Specification

The first screen the user will be met with on starting the program will be the main menu, giving the user the option to:

- Open a window, giving the user a list of tutorials to select from;
- open the simulator in sandbox mode;
- exit the program.

The first two options should allow the user to either begin a new session, or load a logic circuit they have saved previously. Upon opening the simulator the world-space of the simulator will be initialised, and all circuits loaded into the world. At this point the simulator will begin its first pass of the simulation, building a logic-space representation of the circuit. The world-space denotes the visual representation of the circuit that the user will interact with directly, using input devices to build and configure their digital circuit. The world-space representation of the circuit will be grid-based, in which wires and other elements will be aligned to the grid. The logic-space denotes the internal, logical representation of the circuit that will be built by the simulator when the world-space model is modified. This logic-space representation of the circuit should be designed to maximise performance when simulating the circuit. There should be a two-way communication between the logical and world representation of the circuit, so that when a wire changes state in the logical model, the world model updates to lighten or darken the wires based on their activation state.

When initialisation and loading is finished, the user should be presented with the world-space circuit and a graphical interface for allowing the selection of different tools for building and manipulating the circuit. A toolbar at the bottom of the screen will provide the basic logic gates and wires. Other interface elements will control higher functions of the simulator, such as loading and saving circuits. These buttons will be displayed seperately from the toolbar, most likely at the top of the screen as the user would expect. In the case of tutorials, the interface will also include a text box detailing the task the user must complete in order to progress. The selection of logic gates available on the toolbar will be NOT, OR, AND and XOR. The provided wires will come in three colors: red, green and blue.

Users will be able to select a tool from the toolbar. Selecting one of the logic gates will bind a new instance of that logic gate to the mouse cursor. Pressing the left mouse button will place the gate in world-space, while pressing the right mouse button will cancel placement, eliminating the instance of the gate. Selecting one of the wire buttons will enable wire placement, and bring the selected wire's respective layer to the top, rendering it over the top of the other wires. Pressing the left mouse button will place a wire at the current position of the cursor. Holding the

left mouse button down will allow the user to place multiple wires by dragging the mouse. The right mouse button behaves similarly, but instead deletes wires at the current position of the mouse cursor.

If the user has loaded one of the prescribed lessons, the world with initialise with two pre-defined special circuit packages. One will output a series of signals for the user's circuit to transform, and the other will accept the outputs of the user's circuit. If the user's circuit outputs the expected signals, the lesson will be considered complete and the user will be able to move on to the next lesson. A dialog box will appear to offer the option to move on to the next lesson. If the output of the user's circuit is not as expected, this will be made clear to the user though a dialog box.

The user will be able place special input and output nodes and connect them to their circuit. The user should be able to select the circuit and click a button on the interface to convert the selected packages and wires into a circuit package. The input and output nodes will correspond to the pins on the resulting circuit package. This not only allows for the compression of circuits, but also allows the user to easily reuse their circuit. Users will be able to name their input and output nodes, which will be displayed on the circuit package when the circuit is converted. The user-made circuit packages should also be able to be named. Users will be able to manage their saved circuit packages via a menu.

Users will be able to navigate the simulator world via the arrow keys or W, A, S and D. Holding the tab key will make the circuit packages transparent, allowing users to see, place and delete wires underneath circuit packages without having to move them. The mouse scroll-wheel will allow the user to zoom the camera in and out.

In order to make the use of user-made, circuit packages less tedious, the user should be able to open their circuit packages for editing via a context menu. This will open a new file, but also maintain continuity of depth, giving the user a button that allows them to exit out of the sub-circuit back to the parent they accessed it from. This will require the implementation of an additional state stack to hold each parent world's data.

Finally, users should be able to save their circuits to continue work on them at a later time. Saved circuits should be able to be loaded as a circuit package in other sessions. There should be a check here to make sure a user cannot make a circuit include itself recursively. This is also true for any circuit packages that contain the parent circuit, either in itself, or any of its sub-packages.

Project Plan

The plan for the project consists of identifying each module and their responsibilities, how they should interact with one another, and how they should manage and transform data. The first modules to be implemented will be the core modules that facilitate game engine initialisation and state management. The engine works on the principle of a state stack. When the user opens the simulator from the main menu, the simulator state instance is pushed to the top of the state stack. Similarly, when the user exits the simulator to return to the main menu, the simulator state is removed from the stack and the main menu state continues execution. Only the top state on the stack is processed. If the state stack is empty, then the program will terminate. Setting up these states and their three primary functions (handle, update and render) are the second things that should be implemented after the engine is initialised. The rest of the program will extend from these three functions.

Each module that will be called directly from the associated state will follow that same pattern of handle, update, render. Firstly, the world will have to be created. This will contain all the visual elements and tools the user will interact with, wires, circuit packages, etc. This will require a set of world segments of some constant size. World segments will not be rendered or meshed if they are outside of the camera frustum. Each segment will have a flag that will be set when it is to be meshed for rendering. Each segment will hold its own vertex buffer, model matrix, a list of layers which will contain tile data for each wire color, a vertex buffer for those layers (using the same model matrix as the segment itself), and a list of circuit packages that belong to that segment. Circuit packages will be layer independent, so any wire layer can connect to the same circuit package pins. The tile data in each layer will simply be a bit field denoting which sections of the tile have a wire (Up, Down, Left, Right, and Centre). Circuit packages must keep track of their world-space position, a list of inputs, a list of outputs, and the circuit they contain. It must also contain its name, and the names, and positions of its input and output pins. Pre-programmed circuits provided by the software will, instead of pointing to a bound circuit, point to a function that will define its behaviour. This will be especially useful for simulating more complex circuits that would otherwise require a lot of resources to simulate via a logic circuit, such as a memory chip.

The next module to be implemented should be the logic-space representation of the circuit. This will require a separate definition of wires and circuit packages. Connected wires will be packaged as a set of inputs and outputs, which are pointers to pins they connect to. They will have an activation flag; a flag that will be enabled on change of it's activition flag; and a list of which tile coordinates the wire corresponds to. This coordinate list will be used for a two-way

communication channel between the logic-space and world-space representation of the circuit. When the user updates the world-space circuit by adding or removing elements from it, the simulator should update the logic-space circuit to reflect this. The logic-space circuit will start building from package output pins, marking each pin visited when processed. This will mean that only connected wires are checked, and unoccupied or disconnected tiles are never visited. When the logic-space circuit is built, the simulator should make its first processing pass and settle the circuit. Special checks will need to be considered to detect when a circuit is wired in such a way that it continually changes , and never actually finishes the pass, effectively locking the program. A NOT gate connected to itself would cause this issue.

Next, the world must be able to be saved and loaded to and from a file. This will require the development of a file format representing a serialisation of circuit data. A secondary file format will have to be developed that will also contain information required for the lessons. This will contain the input and output test data sets and the text for the lesson brief.

When all this is complete, development can begin on the lessons. This will require careful consideration of what the user has learned at each stage, and what the most logical step forward should be in the next lesson in order to slowly build the users knowledge to a point where they can understand and build a simple computer.

Currently, the project is close to completion of the world-space implementation of circuits. The world, wire creation and deletion, navigation of the world, and the toolbar section of the graphical interface have all been completed. There is a working prototype of this. The next step of development will be to add the world-space circuit packages.

Methodology

In a graphical application such as this, where back-end logic must be performant, but also fast to transform into renderable data, there is a demand for a primary focus on the performance of the application. For this reason, the design for this project will focus on how the data itself should be transformed and less on how it is to be represented structurally. With that, the project will be designed with some data-oriented design principles in mind, along with some object-oriented design patterns where appropriate. The reason for this decision is for speed, especially in parts of the program where a lot data will need to be accessed in series, such as tile states, that could be naturally blocked together in a contiguous memory block. Keeping these packaged together and focusing on how the data relates to each other is good way of producing logical groupings of data that benefit the flow of data in the application, as opposed to trying to relate it as closely as possible to the problem domain. Another reason is that it also allows more flexibility in the case of high-level design changes that may otherwise require a lot of change to the implementation. Additionally, circumstances in which certain stages of the pipeline can be processed concurrently will be more clear, and easier to implement.

Implementation of the project will be done in C. The decision for using this language is multi-faceted. For one, it meshes well with the chosen design paradigm. The game engine and tools that will be used in implementing the application are also written in C. There was some consideration put into using C++, however this would make little difference in terms of how the program is implemented, but certain type-checking standards of the language would become intrusive. Implementation will closely follow the design, while employing some techniques and practices that are common in ensuring good performance, robustness and maintainability: branchless programming, pure functions and flattening, to name a few.

Testing, on a more granular scale, will involve unit tests as the software is developed. The structure of the software and the design principles employed will make this process very simple, so long as the associated structures exist, even in a rudimentary sense. As modules are built and connected to each other, the communication of data between them will be evaluated in terms of correctness and performance. Regular checks will be made for memory leaks and performance issues as the software is developed. At a higher level, later in development, the correctness of the logic simulator will be able to be assessed through the lesson system itself. Using the two input/output circuits to supply and verify test data and output to prove that the circuit simulator operates as intended and is fit for purpose.

In evaluating the software, the primary question that needs answering is “does this software teach users how a computer works.” It will take time to get an honest answer to this

question, and will require participation from people who are not particularly well versed in computational logic. It will not be easy for participants to provide this data, as it will require them to complete the lesson series in its entirety, so some level of interest in the subject will be necessary. It appears possible that, if well thought out, the lessons themselves could act as an evaluation of the user's understanding, and completion of the lessons would be proof of the software's effectiveness in teaching the user how a computer works. Some feedback through a light questionnaire could prove useful in verifying that the software achieves its intended goal, and potentially highlight unforseen issues. Subjecting the software to several different users will also help establish its reliability and robustness.

Each stage of development will make use of a set of tools. The most significant are those used in implementation: An in-house game engine, OpenGL, and SDL. Additional tools are used for evaluating the software's memory management and performance: GDB for debugging; Valgrind for tracking memory leaks; and Heaptrack for additional evaluation of memory metrics.